

VisionInspectAI- Manufacturing Defect Detection & Quality Inspection System

Yash Goyal

Table of Contents

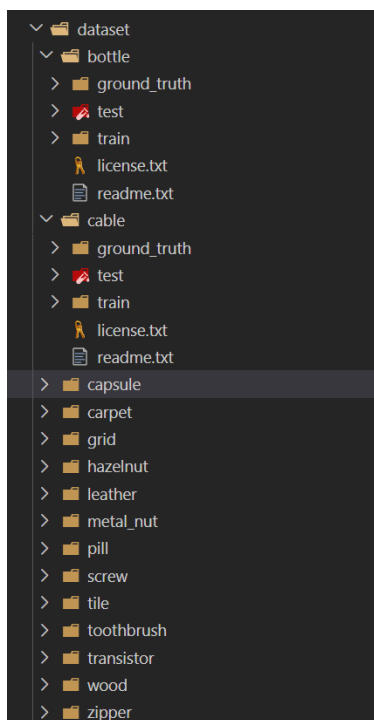
1. Project Description	3
2. Dataset Used	3
3. Environment Setup	4
4. Data Exploration	5
5. Data Preprocessing	6
6. Model Selection, Architecture & Configuration	7
7. AI Pipeline	8-9
8. Model Training	9-10
9. Model Prediction	11-12
10. Conclusion	13

1. Project Description

VisionInspectAI is an AI-powered manufacturing defect detection and quality inspection system that automates the identification of defective products using computer vision and anomaly detection techniques. The system utilizes the PatchCore algorithm from the Anomalib framework to learn the characteristics of normal products and detect anomalies without requiring extensive defect-labeled datasets. For model development and validation, the MVTec AD dataset is used. The current implementation includes dataset exploration, preprocessing, model training, and prediction, providing a foundation for integration with a web-based dashboard, real-time inspection, batch processing, and quality analytics.

2. Dataset Used

The complete MVTec AD dataset containing 15 object categories was added to the project. The dataset includes training images, testing images, and ground truth masks for defective samples.



Dataset Link: <https://www.mvtec.com/research-teaching/datasets/mvtec-ad>

3. Environment Setup

The AI module of **VisionInspect AI** was developed and executed locally using a dedicated Python virtual environment to manage project dependencies and avoid package conflicts. The implementation was carried out using **Python 3.10** and **Visual Studio Code (VS Code)** as the primary development environment. The model training and inference were performed on a Windows-based system using CPU resources.

Development Environment

- Operating System: Windows
- IDE: Visual Studio Code
- Programming Language: Python 3.10
- Virtual Environment: Python venv

Major Libraries Used

- **PyTorch** – Deep learning framework for model execution.
- **Anomalib** – Anomaly detection framework implementing the PatchCore algorithm.
- **Torchvision** – Image preprocessing and pretrained backbone support.
- **OpenCV** – Image loading and preprocessing.
- **NumPy** – Numerical computations.
- **Pandas** – Dataset analysis and manipulation. .
- **Pillow (PIL)** – Image processing utilities.

Hardware Configuration

- Processor: AMD Ryzen CPU
- Memory: 16 GB RAM
- GPU: No dedicated NVIDIA GPU (CPU-based training and inference)

This environment provides a stable and reproducible setup for dataset preprocessing, model training, and anomaly detection using the PatchCore algorithm.

4. Dataset Exploration

The dataset exploration phase was performed to analyze the structure and contents of the **MVTec AD** dataset before preprocessing and model training. A custom Python script (explore_dataset.py) was developed to automatically inspect the dataset and generate useful statistics.

The dataset exploration process includes:

- Verification of the dataset directory and folder structure.
- Identification of all available product categories.
- Counting the total number of training images for each category.
- Counting the number of good and defective test images.
- Identification of different defect types present in each category.
- Counting the number of images corresponding to each defect type.
- Verification of image resolution.
- Verification of image color channels (e.g., 3-channel images).
- Detection of corrupted or unreadable images.
- Generation of an overall dataset summary, including total categories, training images, testing images, good images, defective images, and total images.

This exploration step ensured that the dataset was complete, correctly organized, and ready for the subsequent preprocessing and model training stages.

Outputs:

```
Category : bottle
-----
Training Images      : 209
Good Test Images     : 20
Defective Test Images : 63
Total Test Images    : 83
Ground Truth Masks   : 63
Image Resolution     : 900 x 900
Color Channels       : 3 Channels
Corrupted Images     : 0

Defect Types:
broken_large: 20
broken_small: 22
contamination: 21
```

```
Overall Dataset Summary
Total Categories      : 15
Training Images       : 3629
Testing Images        : 1725
Good Test Images      : 467
Defective Test Images : 1258
Ground Truth Masks    : 1258
Corrupted Images      : 0
Total Images          : 5354
```

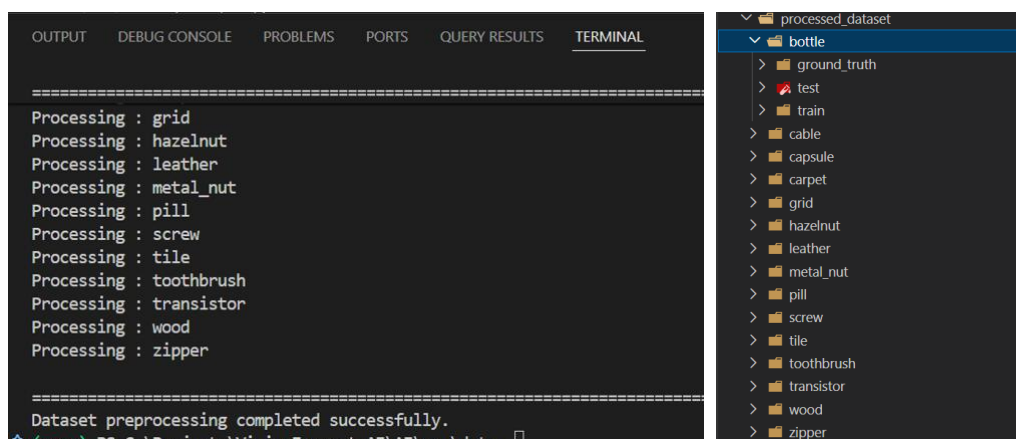
5. Dataset Preprocessing

The preprocessing pipeline standardizes the dataset while preserving the original image content.

- Validated the dataset structure.
- Created the processed dataset directory.
- Read every image.
- Detection and skipping of corrupted or unreadable files.
- Resized images to 256×256 .
- Resized masks using INTER_NEAREST.
- Saved the processed dataset while preserving the original folder structure.

Why Some Operations Were Not Performed

- **RGB Conversion:** Not applied during preprocessing. Original color images were preserved, while PatchCore handles the required image preprocessing internally.
- **Normalization:** Not applied because PatchCore's **built-in PreProcessor** automatically performs ImageNet normalization during training and inference.
- **Data Augmentation:** Not applied because the initial objective was to establish a stable anomaly detection pipeline using the original dataset. Augmentation can be incorporated later during experimentation if required.
- **Noise Reduction:** Skipped as the MVTec AD dataset already contains high-quality images, and denoising could remove subtle defect details important for anomaly detection.
- **CLAHE / Histogram Equalization:** Not applied because the dataset has consistent lighting, and altering image contrast could affect feature extraction by the pretrained model.



6. Model Selection, Architecture and Configuration

Model Selection

The **PatchCore** anomaly detection model from the **Anomalib** framework was selected for this project.

- Designed for industrial anomaly detection.
- Requires only normal images for training.
- identifies anomalies by comparing test images with the learned feature representations.
- Provides anomaly localization through heatmaps.
- Performs strongly on the MVTec AD dataset.
- Officially available in the Anomalib framework.

Model Architecture

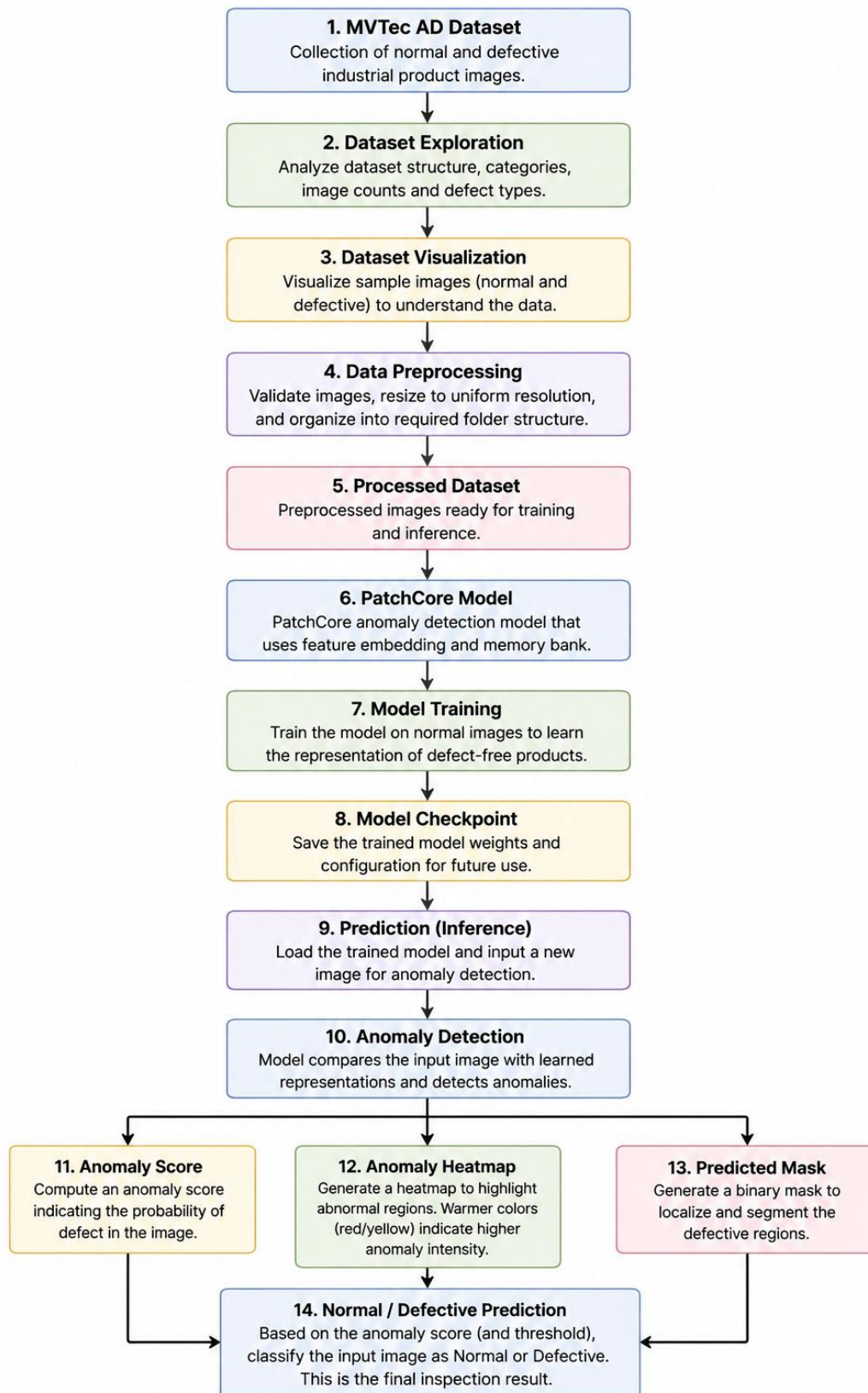
The PatchCore model utilizes a pretrained **Wide ResNet50-2** backbone to extract deep feature representations from input images. Feature maps from **Layer 2** and **Layer 3** are used to capture both low-level texture information and high-level semantic features. During training, PatchCore constructs a **memory bank** using features extracted from normal images. During inference, features from a test image are compared with the memory bank using **Nearest Neighbor Search**. The difference between these features is used to generate an **anomaly score**, which determines whether the image is classified as **Normal** or **Defective**.

Model Configuration

The following configuration was used for the current implementation:

Parameter	Value
Model	PatchCore
Framework	Anomalib
Backbone	Wide ResNet50-2
Feature Layers	Layer2, Layer3
Coreset Sampling Ratio	0.1
Number of Nearest Neighbors	9
Pretrained Backbone	Yes
Input Image Size	256 × 256
Batch Size (Training)	32
Batch Size (Evaluation)	32

7. AI Pipeline (Currently Implemented)



Note1: *In the complete VisionInspect AI system, this pipeline will be extended with additional modules such as **defect classification, confidence estimation, severity scoring, and inspection reporting**. These components are planned for future milestones.*

Note2: *Although not included in the Milestone 1 deliverables, the following work has also been completed to validate the AI pipeline:*

- Successfully configured the PatchCore anomaly detection model.
- Trained the model on the **Bottle** category of the MVTec AD dataset.
- Generated and stored the trained model checkpoint.
- Developed the prediction module for single-image inference.
- Successfully tested the model on both normal and defective bottle images.
- Verified that the model correctly classifies images and generates anomaly scores.

8. Model Training(Category-Bottle)

The model training process was carried out using the **PatchCore** anomaly detection model on the **Bottle** category of the MVTec AD dataset to validate the complete AI pipeline.

Model Training Process

- The processed dataset was loaded using the **MVTec AD DataModule**.
- Only **normal (defect-free)** training images were used to train the PatchCore model.
- A pretrained **Wide ResNet50-2** backbone was used for feature extraction.
- Deep feature embeddings from **Layer 2** and **Layer 3** were extracted.
- A **memory bank** was constructed using the extracted feature representations of normal images.
- **Coreset sampling** was applied to reduce memory usage while preserving representative features.
- The trained model was saved as a **checkpoint (.ckpt)** for future inference.
- The saved checkpoint is later loaded during the prediction stage to detect anomalies in unseen images.

Training Screenshots:

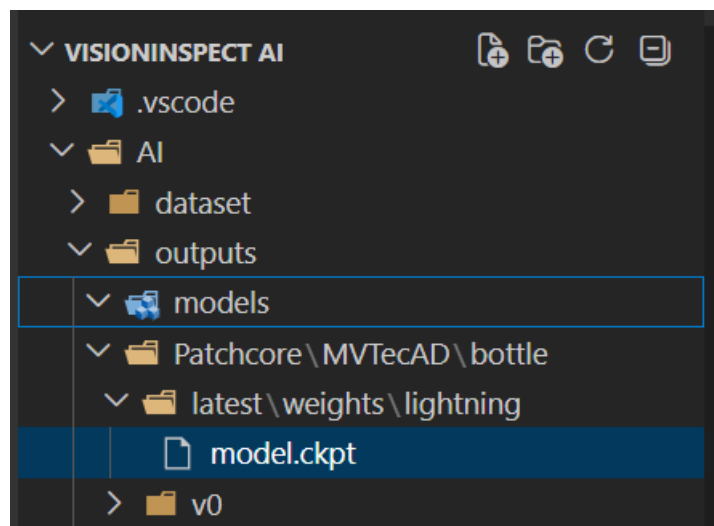
	Name	Type	Params	Mode	FLOPs
0	pre_processor	PreProcessor	0	train	0
1	post_processor	PostProcessor	0	train	0
2	evaluator	Evaluator	0	train	0
3	model	PatchcoreModel	24.9 M	train	0

Trainable params: 24.9 M
Non-trainable params: 0
Total params: 24.9 M
Total estimated model params size (MB): 99.450
Modules in train mode: 19
Modules in eval mode: 174

```
OUTPUT  DEBUG CONSOLE  PROBLEMS  PORTS  QUERY RESULTS  TERMINAL
Selecting Coreset Indices.: 100% [19:03:00:01, 17.67it/s] 21379/21400
Selecting Coreset Indices.: 100% [19:03:00:01, 17.46it/s] 21381/21400
Selecting Coreset Indices.: 100% [19:04:00:00, 17.42it/s] 21383/21400
Selecting Coreset Indices.: 100% [19:04:00:00, 17.43it/s] 21385/21400
Selecting Coreset Indices.: 100% [19:04:00:00, 18.11it/s] 21387/21400
Selecting Coreset Indices.: 100% [19:04:00:00, 18.39it/s] 21389/21400
Selecting Coreset Indices.: 100% [19:04:00:00, 18.52it/s] 21391/21400
Selecting Coreset Indices.: 100% [19:04:00:00, 17.96it/s] 21393/21400
Selecting Coreset Indices.: 100% [19:04:00:00, 17.25it/s] 21395/21400
Selecting Coreset Indices.: 100% [19:04:00:00, 17.06it/s] 21397/21400
Selecting Coreset Indices.: 100% [19:04:00:00, 17.07it/s] 21399/21400
Selecting Coreset Indices.: 100% [19:05:00:00, 18.69it/s] 21400/21400

Epoch 0/0 7/7 0:00:27 * 0:00:00 0.26it/s `Trainer.fit` stopped: `max_epochs=1` reached.
Epoch 0/0 7/7 0:00:27 * 0:00:00 0.26it/s
(venv) PS C:\Projects\VisionInspect AI\AI\src>
```

Checkpoint Saved:



9. Model Prediction

The prediction module was developed to evaluate new product images using the trained **PatchCore** model. It loads the saved model checkpoint and performs anomaly detection on input images to determine whether a product is **Normal** or **Defective**.

Model Prediction Process

- The trained **PatchCore** model is loaded from the saved checkpoint.
- A user provides an input image (or image folder) for inspection.
- The input image is automatically preprocessed using PatchCore's built-in **PreProcessor**.
- The model extracts deep feature representations from the input image.
- The extracted features are compared with the memory bank created during training.
- An **anomaly score** is computed to measure the likelihood of a defect.
- Based on the anomaly score, the image is classified as **Normal** or **Defective**.
- An **anomaly heatmap** is generated to highlight abnormal regions in the image.
- A **predicted defect mask** is generated to localize the detected defect.
- The prediction results, including the anomaly score, prediction label, heatmap, and defect mask, are saved for visualization and further analysis.

Challenge Faced During Prediction:

At first time, Model was predicting wrong, a good category- defective one due to duplicate normalization because normalization is explicitly defined in datamodule.py but PatchCore already performs ImageNet normalization. Hence, we removed duplicate normalization and trained the model again.

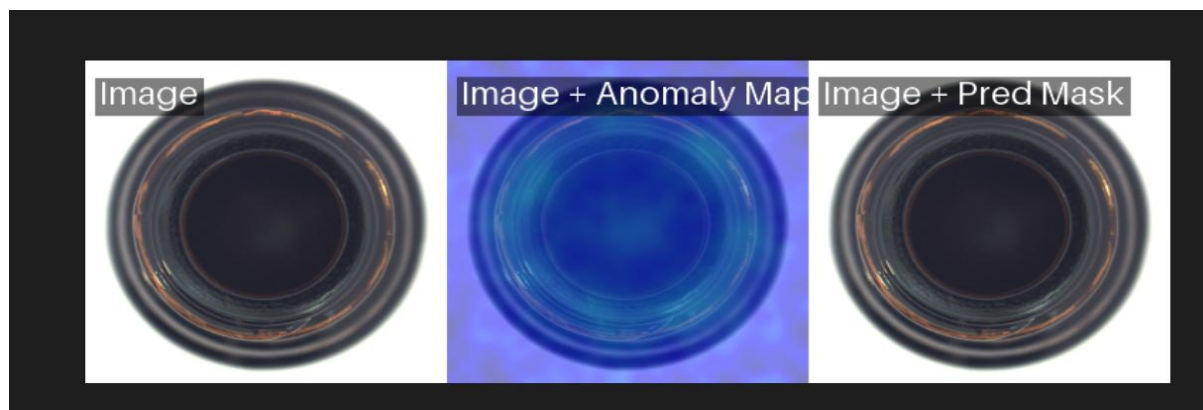
Prediction Outputs:

1. Good Category image as input:

```
(venv) PS C:\Projects\VisionInspect AI\AI\src> python -m models.predict
ated, use `isinstance(treespec, TreeSpec)` and `treespec.is_leaf()` instead.
C:\Projects\VisionInspect AI\venv\lib\site-packages\lightning\pytorch\trainer\connectors\data_connector.py:434: The 'predict_dataloader'
does not have many workers which may be a bottleneck. Consider increasing the value of the 'num_workers' argument to 'num_workers=11' in
the 'DataLoader' to improve performance.
C:\Projects\VisionInspect AI\venv\lib\site-packages\torch\utils\data\loader.py:752: UserWarning: 'pin_memory' argument is set as true
but no accelerator is found, then device pinned memory won't be used.
  super().__init__(loader)
Predicting 1/1 0:00:00 = 0:00:00 0.00it/s

Image      : 000.png
Category   : bottle
Status     : Normal
Anomaly Score : 0.2267

(venv) PS C:\Projects\VisionInspect AI\AI\src>
```



2. Defective Category image as input:

```

OUTPUT  DEBUG CONSOLE  PROBLEMS  PORTS  QUERY RESULTS  TERMINAL
powershell - src + v [ ] [ ] ... | [ ] x

(venv) PS C:\Projects\VisionInspect AI\AI\src> python -m models.predict
ated, use `isinstance(treespec, TreeSpec)` and `treespec.is_leaf()` instead.
C:\Projects\VisionInspect AI\venv\lib\site-packages\lightning\pytorch\trainer\connectors\data_connector.py:434: The 'predict_dataloader'
does not have many workers which may be a bottleneck. Consider increasing the value of the `num_workers` argument` to `num_workers=11` in
the `DataLoader` to improve performance.
C:\Projects\VisionInspect AI\venv\lib\site-packages\torch\utils\data\dataloader.py:752: UserWarning: `pin_memory` argument is set as true
but no accelerator is found, then device pinned memory won't be used.
  super().__init__(loader)
Predicting 1/1 0:00:01 • 0:00:00 0.00it/s

Image      : 000.png
Category   : bottle
Status     : Defective
Anomaly Score : 0.942

(venv) PS C:\Projects\VisionInspect AI\AI\src>

```

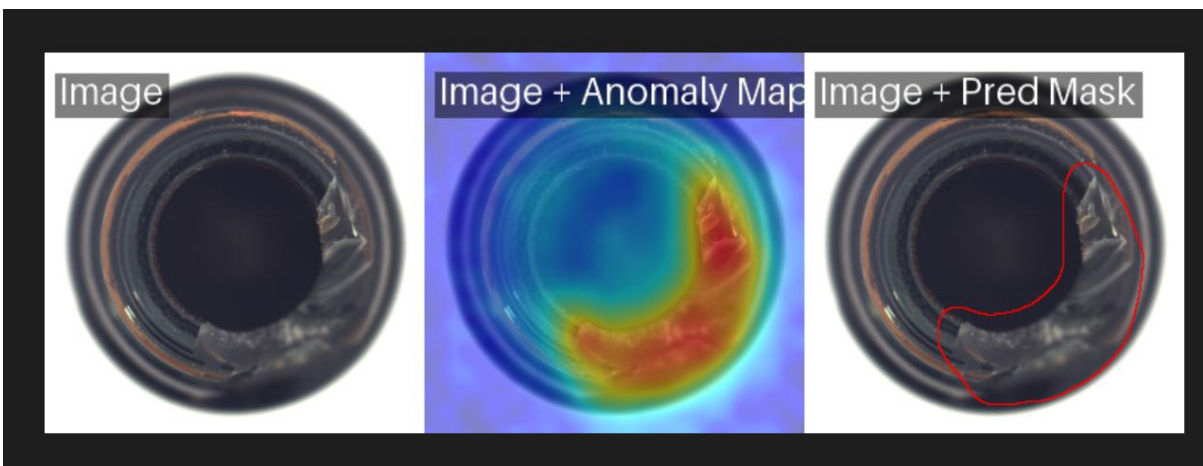


Figure: Prediction results showing the original image, anomaly heatmap, and predicted defect mask.

10.Conclusion

The first phase of the **VisionInspectAI** project successfully established the foundation for an AI-based manufacturing defect detection system. The MVTec AD dataset was explored, visualized, and preprocessed to ensure it was suitable for model development. A structured AI pipeline was designed, and the PatchCore anomaly detection model was selected and configured for industrial defect detection.

Although **model training and prediction** are scheduled for **Milestone 2**, a **preliminary implementation** was completed using the **Bottle** category to **validate the end-to-end workflow**. The trained model successfully generated anomaly scores, prediction labels, anomaly heatmaps, and predicted defect masks, demonstrating the effectiveness of the implemented pipeline.

The completed work provides a strong foundation for the next phase of the project, which will focus on extending the model to all dataset categories and integrating advanced features such as **defect classification, severity scoring, inspection reporting**, and a **web-based quality inspection system**.